Exp 11:

Aim:

To calculate the ε-closure of each state in an NFA with ε-transitions.

```c
#include <stdio.h>

int main() {
    int e[3][3] = {
        {0, 1, 1},
        {0, 0, 0},
        {0, 0, 0}
    };
    int i, j, k, c[3][3] = {0};

    for (i = 0; i < 3; i++)
        c[i][i] = 1;

    for (k = 0; k < 3; k++)
        for (i = 0; i < 3; i++)
            for (j = 0; j < 3; j++)
                c[i][j] |= c[i][k] & e[k][j];

    for (i = 0; i < 3; i++) {
        printf("E-closure(q%d) = {", i);
        for (j = 0; j < 3; j++)
            if (c[i][j]) printf(" q%d", j);
        printf(" }\n");
    }

    return 0;
}
```

```
E-closure(q0) = { q0 q1 q2 }
E-closure(q1) = { q1 }
E-closure(q2) = { q2 }
```

Result :

Thus, the ε-closure of all states was calculated successfully.

Exp 12:

Aim:

To design a DFA that accepts exactly the strings a, ac, and bac.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];

    printf("Enter a string: ");
    scanf("%99s", s);

    if (!strcmp(s, "a") ||
        !strcmp(s, "ac") ||
        !strcmp(s, "bac"))
        printf("String accepted");
    else
        printf("String rejected");

    return 0;
}
```

```
Enter a string: a
String accepted
```

Result :

Thus, the DFA language was implemented successfully. The strings a, ac, and bac are accepted.

Exp 13:

Aim:

To simulate a PDA that accepts the string aabb by matching two a symbols with two b symbols.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i, n, count = 0, ok = 1;

    printf("Enter a string: ");
    scanf("%99s", s);
    n = strlen(s);

    for (i = 0; i < n; i++) {
        if (s[i] == 'a' && count >= 0)
            count++;
        else if (s[i] == 'b' && count > 0)
            count--;
        else {
            ok = 0;
            break;
        }
    }

    if (ok && count == 0 && n == 4)
        printf("String accepted");
    else
        printf("String rejected");

    return 0;
}
```

```
Enter a string: aabb
String accepted
```

Result :

Thus, the PDA simulation was implemented successfully. The input aabb is accepted.

Exp 14 :

Aim :

To simulate a PDA that accepts strings containing n occurrences of a followed by 2n occurrences of b.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i = 0, a = 0, b = 0, phase = 0, ok = 1;

    printf("Enter a string: ");
    scanf("%99s", s);

    while (s[i]) {
        if (s[i] == 'a' && phase == 0)
            a++;
        else if (s[i] == 'b') {
            phase = 1;
            b++;
        } else {
            ok = 0;
            break;
        }
        i++;
    }

    if (ok && a > 0 && b == 2 * a)
        printf("String accepted");
    else
        printf("String rejected");

    return 0;
}
```

```
Enter a string: abbbb
String rejected
```

Result :

Thus, the PDA simulation was implemented successfully. For example, abb and aabbbb are accepted.

Exp 15 :

Aim :

To simulate a Turing Machine that accepts strings containing equal numbers of a symbols followed by b symbols.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i, a = 0, b = 0, phase = 0, ok = 1;

    printf("Enter a string: ");
    scanf("%99s", s);

    for (i = 0; s[i]; i++) {
        if (s[i] == 'a' && phase == 0)
            a++;
        else if (s[i] == 'b') {
            phase = 1;
            b++;
        } else {
            ok = 0;
            break;
        }
    }

    if (ok && a > 0 && a == b)
        printf("String accepted");
    else
        printf("String rejected");

    return 0;
}
```

```
Enter a string: ab
String accepted
```

Result :

Thus, the Turing Machine language was implemented successfully. For example, ab and aabb are accepted.

Exp 16 :

Aim :

To simulate a Turing Machine that accepts strings containing n occurrences of a followed by 2n occurrences of b.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i, a = 0, b = 0, phase = 0, ok = 1;

    printf("Enter a string: ");
    scanf("%99s", s);

    for (i = 0; s[i]; i++) {
        if (s[i] == 'a' && phase == 0)
            a++;
        else if (s[i] == 'b') {
            phase = 1;
            b++;
        } else {
            ok = 0;
            break;
        }
    }

    if (ok && a > 0 && b == 2 * a)
        printf("String accepted");
    else
        printf("String rejected");

    return 0;
}
```

```
Enter a string: abb
String accepted
```

Result :

Thus, the Turing Machine language was implemented successfully. The strings abb and aabbbb are accepted.

Exp 17 :

Aim :

To simulate a Turing Machine that checks whether a string is a palindrome and accepts ababa.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i, j, ok = 1;

    printf("Enter a string: ");
    scanf("%99s", s);

    i = 0;
    j = strlen(s) - 1;

    while (i < j) {
        if (s[i] != s[j]) {
            ok = 0;
            break;
        }
        i++;
        j--;
    }

    if (ok)
        printf("String accepted");
    else
        printf("String rejected");

    return 0;
}
```

```
Enter a string: ababa
String accepted
```

Result :

Thus, the Turing Machine simulation was implemented successfully. The string ababa is accepted.

Exp 18 :

Aim :

To check whether an input string consists of two identical consecutive halves, represented by ww

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i, n, ok = 1;

    printf("Enter a string: ");
    scanf("%99s", s);

    n = strlen(s);

    if (n % 2 != 0)
        ok = 0;
    else {
        for (i = 0; i < n / 2; i++) {
            if (s[i] != s[i + n / 2]) {
                ok = 0;
                break;
            }
        }
    }

    if (ok)
        printf("String accepted");
    else
        printf("String rejected");

    return 0;
}
```

```
Enter a string: abab
String accepted
```

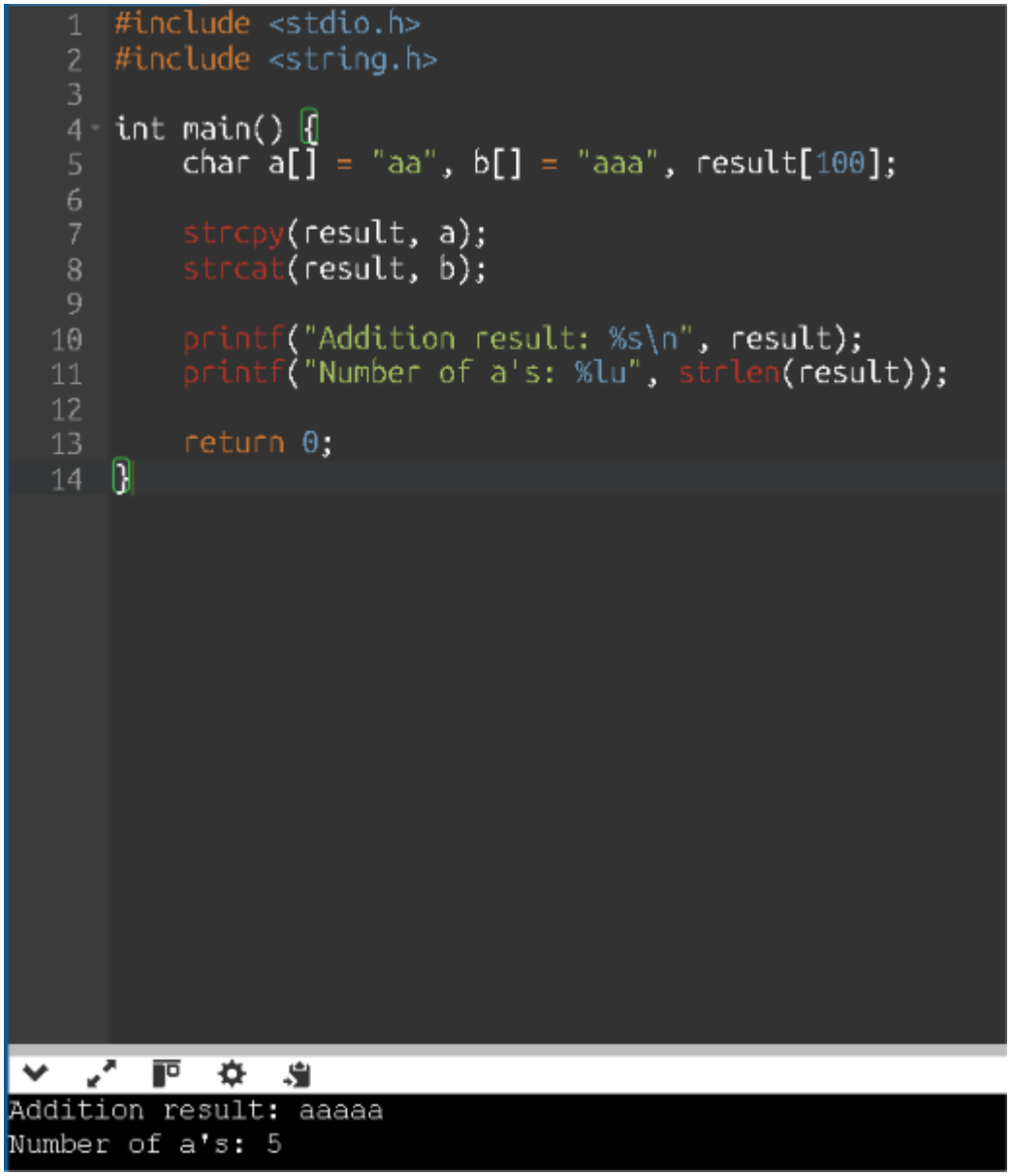
Result :

Thus, the string comparison was implemented successfully. The string abab is accepted because w = ab.

Exp 19 :

Aim :

To simulate unary addition by combining two unary strings, aa and aaa.



```c
#include <stdio.h>
#include <string.h>

int main() {
    char a[] = "aa", b[] = "aaa", result[100];

    strcpy(result, a);
    strcat(result, b);

    printf("Addition result: %s\n", result);
    printf("Number of a's: %lu", strlen(result));

    return 0;
}
```

```
Addition result: aaaaa
Number of a's: 5
```

Result :

Thus, unary addition was performed successfully.

Exp 20 :

Aim :

To simulate unary subtraction by subtracting aa from aaa.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char a[] = "aaa", b[] = "aa";
    int x = strlen(a), y = strlen(b);

    if (x >= y) {
        printf("Subtraction result: ");
        for (int i = 0; i < x - y; i++)
            printf("a");
        printf("\nRemaining a's: %d", x - y);
    } else {
        printf("Negative result");
    }

    return 0;
}
```

```
Subtraction result: a
Remaining a's: 1
```

Result :

Thus, unary subtraction was performed successfully.